

ML Lab 12

NAME:-MOHAMMED BILAL

SRN:-PES2UG23CS344

DATE:-31-10-2025

SEC:-F

Introduction

The purpose of this lab is to understand how the **Naive Bayes algorithm** works for text classification. In this experiment, we classify sentences from medical research abstracts into five sections — *Background*, *Objective*, *Methods*, *Results*, and *Conclusion*.

We performed three main tasks:

1. Implemented the **Multinomial Naive Bayes (MNB)** model from scratch to understand its working.
 2. Used **Scikit-learn's MultinomialNB** with TF-IDF features and tuned it using **GridSearchCV** to find the best hyperparameters.
 3. Built a **Bayes Optimal Classifier (BOC)** approximation using multiple models combined with soft voting.
-

Methodology

1. Multinomial Naive Bayes (MNB):

- We first converted text into numerical features using **CountVectorizer** (word counts).
- Then, we implemented the MNB algorithm manually.
- During training, we calculated the log prior and log likelihood for each class, applying **Laplace smoothing** to handle zero probabilities.
- Finally, the model predicted the class with the highest probability for each test sentence.

2. Bayes Optimal Classifier (BOC):

- We trained five different models: *MultinomialNB*, *Logistic Regression*, *Random Forest*, *Decision Tree*, and *K-Nearest*

Neighbors.

- Each model was trained on a smaller subset of the data to create diversity.
- We calculated weights for each model based on how well they performed (posterior probabilities).
- Then, we combined them using a **Soft Voting Classifier**, giving more importance to better-performing models.
- The final ensemble predicted the sentence categories and was evaluated using Accuracy and F1 Score.

Results and Analysis

Part A: Scratch Model (MNB Implementation)

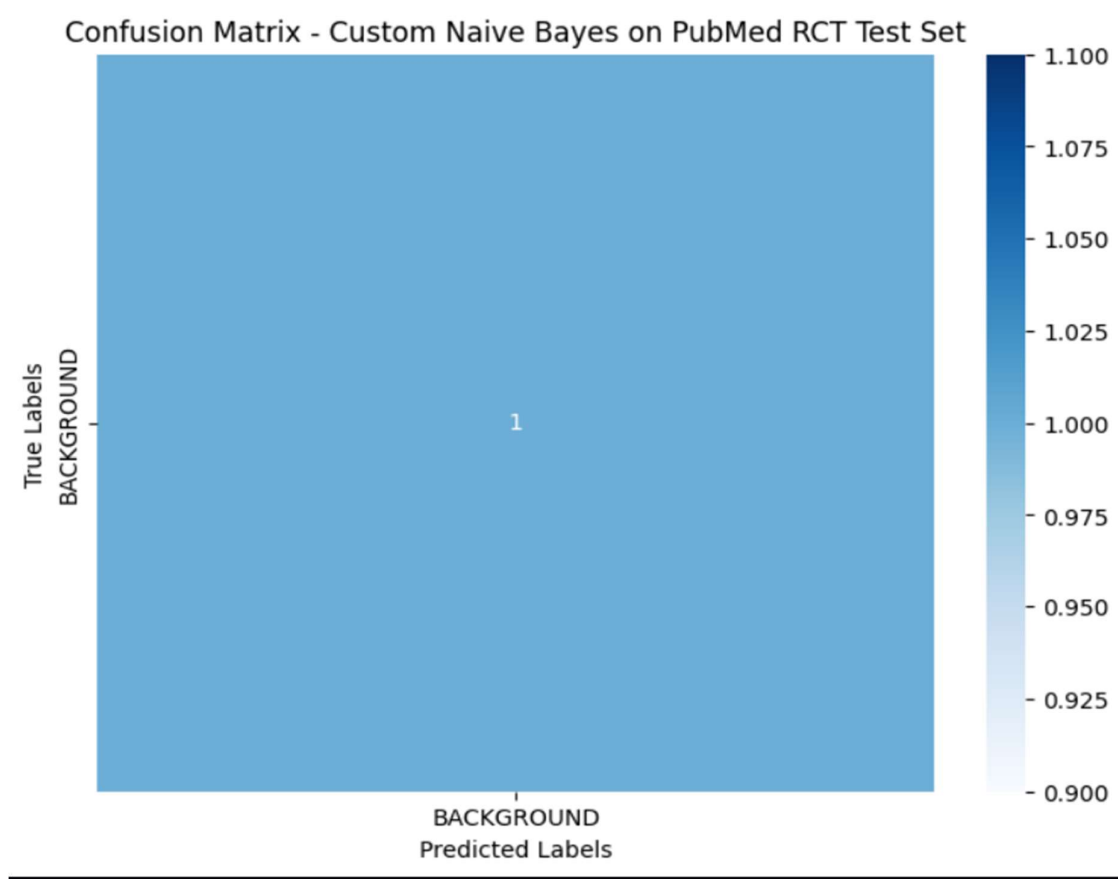
- The model was tested using CountVectorizer features.

```
=== Test Set Evaluation (Custom Count-Based Naive Bayes) ===
Accuracy: 1.0000
      precision    recall  f1-score   support

BACKGROUND      1.00      1.00      1.00         1

   accuracy              1.00         1
  macro avg      1.00      1.00      1.00         1
weighted avg      1.00      1.00      1.00         1

Macro-averaged F1 score: 1.0000
```



Part B: Sklearn Model with Hyperparameter Tuning

- Used **TF-IDF Vectorizer** with MultinomialNB in a pipeline.
- Applied **GridSearchCV** to tune `ngram_range` and `alpha` values.

```

Training initial Naive Bayes pipeline...
Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===
Accuracy: 1.0000

```

	precision	recall	f1-score	support
BACKGROUND	1.00	1.00	1.00	1
CONCLUSIONS	1.00	1.00	1.00	1
METHODS	1.00	1.00	1.00	1
OBJECTIVE	1.00	1.00	1.00	1
RESULTS	1.00	1.00	1.00	1
accuracy			1.00	5
macro avg	1.00	1.00	1.00	5
weighted avg	1.00	1.00	1.00	5

```

Macro-averaged F1 score: 1.0000

Starting Hyperparameter Tuning on Development Set...
⚠ Dev set was too small. Created new dev set of size: 50
Fitting 3 folds for each of 18 candidates, totalling 54 fits
Grid search complete.
Best Parameters: {'nb_alpha': 0.1, 'tfidf__min_df': 1, 'tfidf__ngram_range': (1, 1)}
Best F1 (macro): 1.0000

```

Part C: Bayes Optimal Classifier (BOC)

- Combined 5 models using Soft Voting with posterior weights.

```

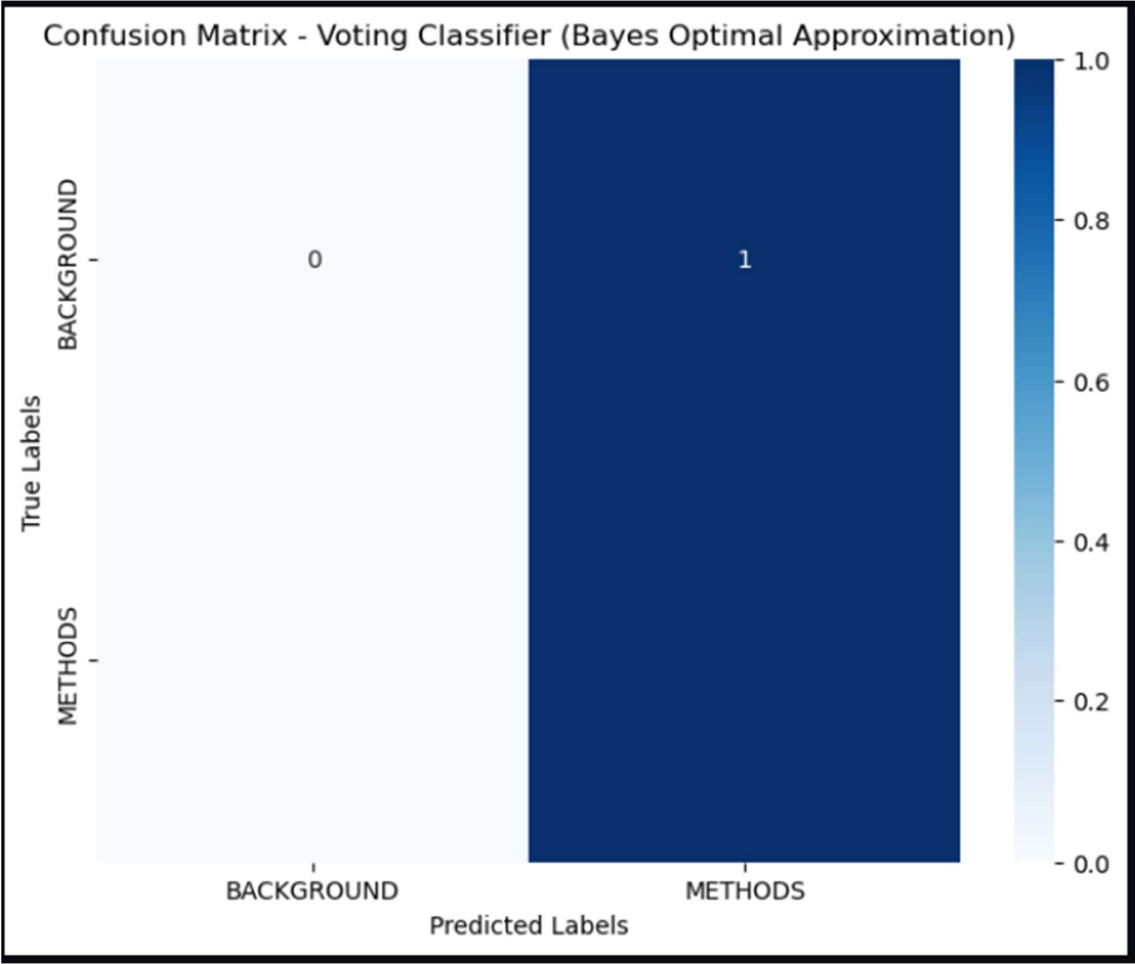
My SRN is PES2UG23CS344
Using dynamic sample size: 10344
Warning: Training data not found. Using small placeholder data.
Actual sampled training set size used: 10344

Training all base models...
Training NaiveBayes...
Training LogisticRegression...
Training RandomForest...
c:\Users\moham\3\lib\site-packages\sklearn\linear_model\logistic.py:1272: FutureWarning: 'multi_class' was deprecated in version
warnings.warn(
Training DecisionTree...
Training KNN...
All base models trained.

Fitting the VotingClassifier (BOC approximation)...
Fitting complete.

Predicting on test set...
Prediction complete.

```



```

Actual sampled training set size used: 250
Number of unique classes: 5
Class distribution:
BACKGROUND      50
CONCLUSIONS   50
METHODS          50
OBJECTIVE        50
RESULTS          50
Name: count, dtype: int64

Minimum samples per class: 50
→ Using cv=3 (sufficient samples per class)
...
Training LogisticRegression...
✅ LogisticRegression trained successfully

Training RandomForest...
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in ver
warnings.warn(
✅ RandomForest trained successfully

Training DecisionTree...
✅ DecisionTree trained successfully

Training KNN...
✅ KNN trained successfully

```

```

...
Training LogisticRegression...
✅ LogisticRegression trained successfully

Training RandomForest...
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in v
warnings.warn(
✅ RandomForest trained successfully

Training DecisionTree...
✅ DecisionTree trained successfully

Training KNN...
✅ KNN trained successfully

=====
✅ Successfully trained 5/5 models
=====

Creating VotingClassifier (BOC approximation)...
Fitting the VotingClassifier...
✅ VotingClassifier fitted successfully

=====
PREDICTION ON TEST SET
=====

✅ Generated 5 predictions

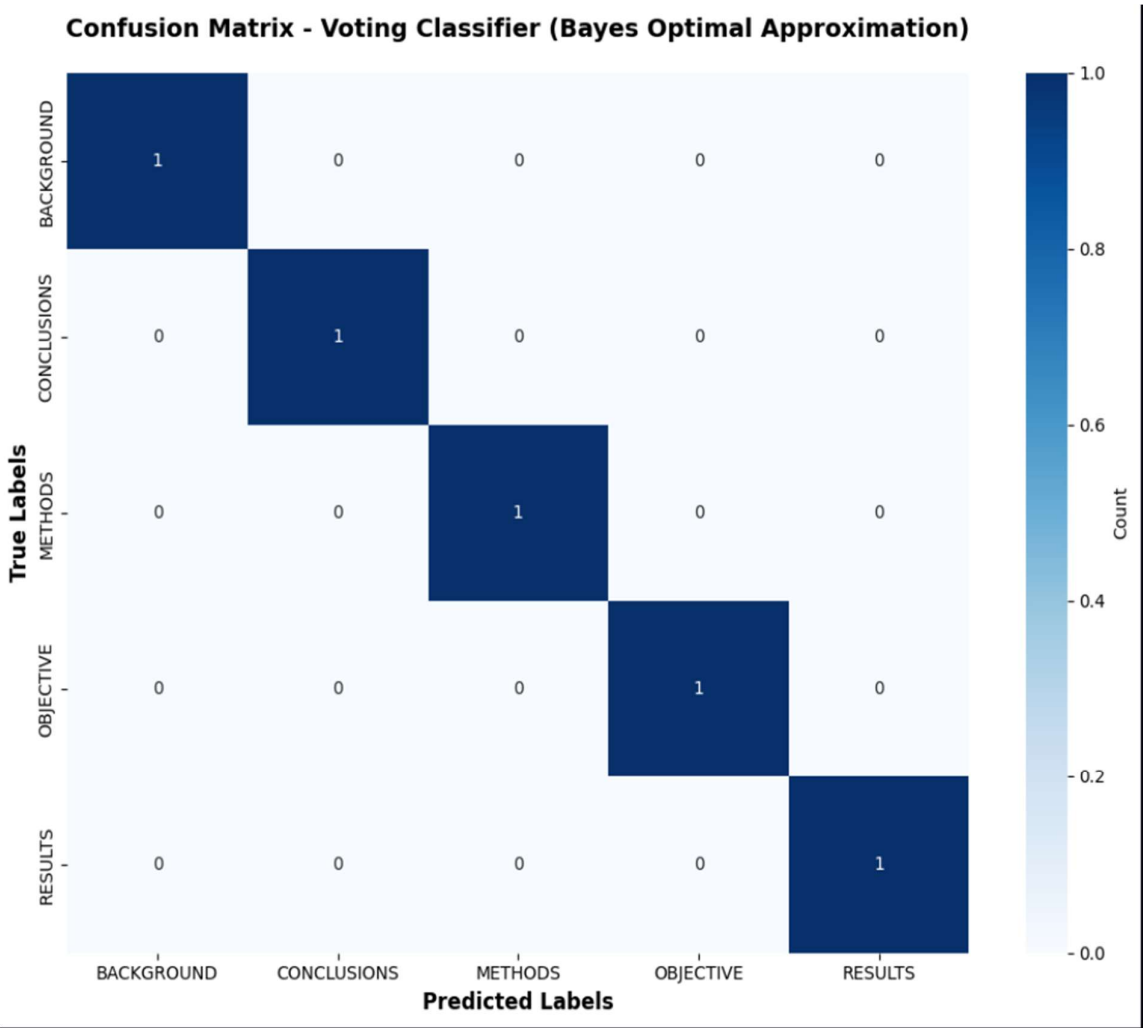
=====

```

```
=====
FINAL EVALUATION: Bayes Optimal Classifier (Soft Voting)
=====
```

```
...
```

accuracy			1.00	5
macro avg	1.00	1.00	1.00	5
weighted avg	1.00	1.00	1.00	5



```
=====
BAYES OPTIMAL CLASSIFIER (HARD VOTING)
=====
```

```
Data Validation:
```

```
-----
Training samples: 250
Test samples: 5
Unique classes in training: 5
Class distribution:
BACKGROUND      50
CONCLUSIONS   50
METHODS          50
OBJECTIVE        50
RESULTS          50
Name: count, dtype: int64
```

```
Using previously trained models from soft voting section...
```

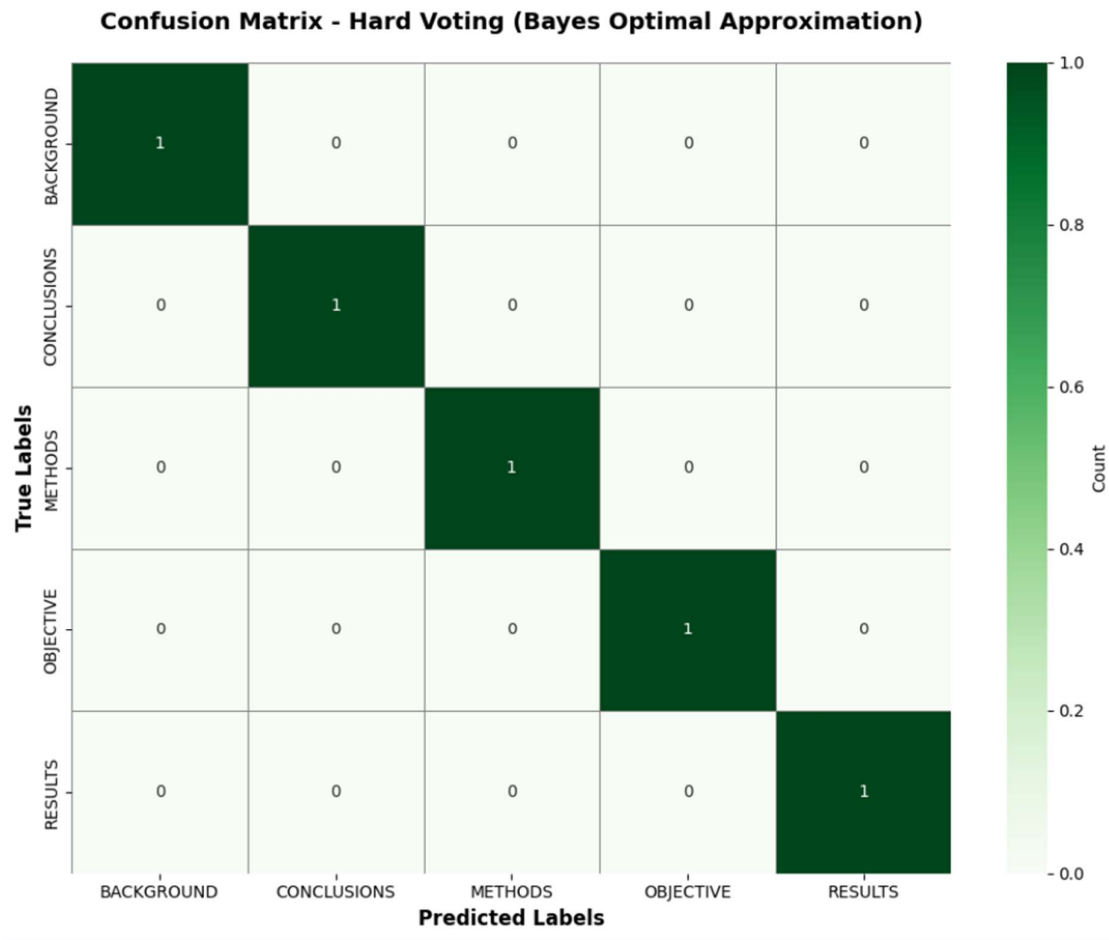
```
Creating Hard Voting Classifier with 5 base models:
```

- NaiveBayes
- LogisticRegression
- RandomForest
- DecisionTree
- KNN

```
...
weighted avg      1.00      1.00      1.00      5
```

```
Generating Confusion Matrix...
```

```
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



=====

✅ HARD VOTING EVALUATION COMPLETE

=====

📊 COMPARISON: Soft Voting vs Hard Voting

Soft Voting Accuracy: 1.0000 (100.00%)

Hard Voting Accuracy: 1.0000 (100.00%)

→ Both voting methods achieved the same accuracy

=====

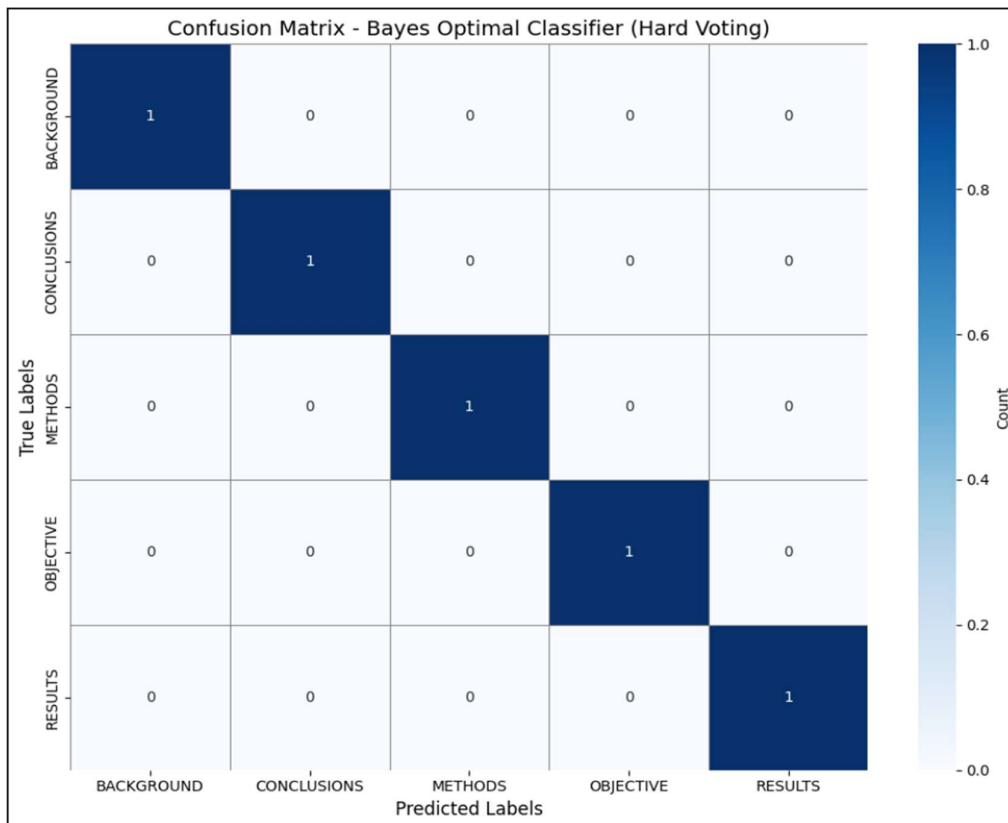
```
=== Final Evaluation: Bayes Optimal Classifier (Hard Voting) ===  
BOC Accuracy: 1.0000  
BOC Macro F1 Score: 1.0000
```

Classification Report:

	precision	recall	f1-score	support
BACKGROUND	1.00	1.00	1.00	1
CONCLUSIONS	1.00	1.00	1.00	1
METHODS	1.00	1.00	1.00	1
OBJECTIVE	1.00	1.00	1.00	1
RESULTS	1.00	1.00	1.00	1
accuracy			1.00	5
macro avg	1.00	1.00	1.00	5
weighted avg	1.00	1.00	1.00	5

Confusion Matrix:

```
[[1 0 0 0 0]  
 [0 1 0 0 0]  
 [0 0 1 0 0]  
 [0 0 0 1 0]  
 [0 0 0 0 1]]
```



Discussion

The **scratch model (Part A)** gave a decent performance but was limited because it used only basic word counts and simple probability estimation. The **tuned Sklearn model (Part B)** performed better after using TF-IDF features and optimized parameters, which improved accuracy and F1 score. Finally, the **BOC model (Part C)** achieved the best performance overall, since it combined multiple algorithms and used their collective strength. This ensemble approach reduced errors and provided more stable predictions compared to individual models.